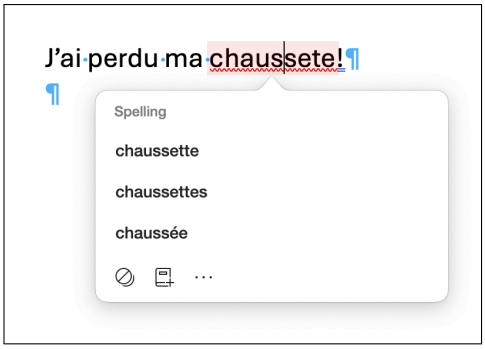
Labo 4: Correcteur orthographique

206.3 Formal and Natural Languages (FNL)



Introduction, Objectifs

Dans ce labo, nous allons implémenter un correcteur orthographique en utilisant un algorithme de correction d'erreurs. Les objectifs sont de :

- comprendre comment fonctionne un correcteur orthographique
- implémenter un correcteur orthographique en Python
- comprendre et appliquer les concepts d'évaluation, de corpus et de vocabulaire
- évaluer les performances de différentes versions d'un correcteur orthographique

Le temps de travail estimé pour ce labo est de 2 heures. Vous réaliserez ce labo par groupes de 2 personnes au maximum. Pour ce travail, vous pouvez utiliser les LLMs pour poser des questions, mais pas résoudre ni écrire le code.

Partie 1: Fonctionnement d'un correcteur orthographique

Un correcteur orthographique est un algorithme qui prend en entrée un mot et qui retourne une liste de mots proches, par exemple :

```
correct_spelling("chaussete")  # attendu: ["chaussette", "chaussettes"]
```

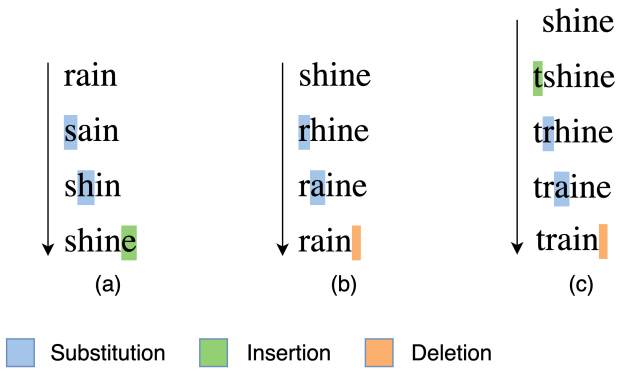
(ici, le mot correct est "chaussette". Il est obtenu en ajoutant la lettre "t" à "chaussete")

```
correct_spelling("chausssette")  # attendu: ["chaussette", "chaussettes"]
```

(ici, le mot correct est obtenu en supprimant un "s" de "chausssette")

Voici les opérations que nous allons utiliser pour "corriger" un mot :

- Ajouter une lettre
- Supprimer une lettre
- Remplacer une lettre
- Échanger deux lettres contiguës



Dans la figure ci-dessus, nous pouvons ainsi passer du mot “rain” au mot “shine” à l’aide de 3 *edits*: deux substitutions et une insertion. On parle de *distance de Levenshtein* entre deux mots.

Question: combien faut-il d’*edits* pour passer du mot “chocolat” au mot “koala”?

Partie 2: Implémentation des opérations

Ecrivez dans un fichier `spelling_correction.py` les différentes fonctions qui permettent d’effectuer les opérations décrites ci-dessus.

```python
ALPHABET = "abcdefghijklmnopqrstuvwxyz"

def ajouter_lettre(mot) -> set[str]:
    """Retourne une liste de mots obtenus en ajoutant chaque lettre de l'
       alphabet à n'importe quelle position du mot."""
    pass

def supprimer_lettre(mot) -> set[str]:
    """Retourne une liste de mots obtenus en supprimant une lettre du mot à n'
       importe quelle position."""
    pass

def substituer_lettre(mot) -> set[str]:
    """Retourne une liste de mots obtenus en remplaçant chaque lettre du mot
       par une autre lettre de l'alphabet."""
    pass

def transposer_lettres(mot) -> set[str]:
    """Retourne une liste de mots obtenus en échangeant deux lettres contiguës
       dans le mot."""
    pass
```

Vous trouverez des tests unitaires dans le fichier `test_spelling_correction.py`.

utilisez une approche fonctionnelle à l’aide de list comprehensions, par exemple :
`return [... for lettre in ALPHABET for i in range(len(mot))]`

Agrégation des opérations

Ecrivez dans votre fichier `spelling_correction.py` une fonction `edits1(mot)-> set[str]` qui prend en entrée un mot et qui retourne un ensemble de mots obtenus en appliquant toutes les opérations à partir de

la liste des opérations. Ecrivez dans `test_spelling_correction.py` un test unitaire pour vérifier que pour le mot `"chaussete"`, votre fonction `edits1` retourne 492 edits possibles. Votre test doit aussi vérifier que le mot `"chaussette"` est dans la liste des edits.

Partie 3: Correction d'un mot

Reste maintenant la question: comment choisir la "meilleure" correction d'un mot? En d'autres termes, parmi les 492 édits possibles pour le mot `"chaussete"`, lequel choisir? Voilà quelques approches que l'on pourrait envisager:

- Une approche simple consisterait à vérifier lequels de ces mots apparaissent le plus fréquemment dans un dictionnaire de la langue française. Le désavantage de cette approche est que le mot correct n'est pas forcément dans le dictionnaire, et qu'il faut trouver un dictionnaire et le mettre à jour régulièrement.
- Une autre approche consiste à construire un dictionnaire de mots à partir d'un large corpus de texte comme Wikipédia. Cette approche est plus efficace, et nous allons l'utiliser dans cette partie.

Construction d'un dictionnaire de mots à partir de Wikipédia

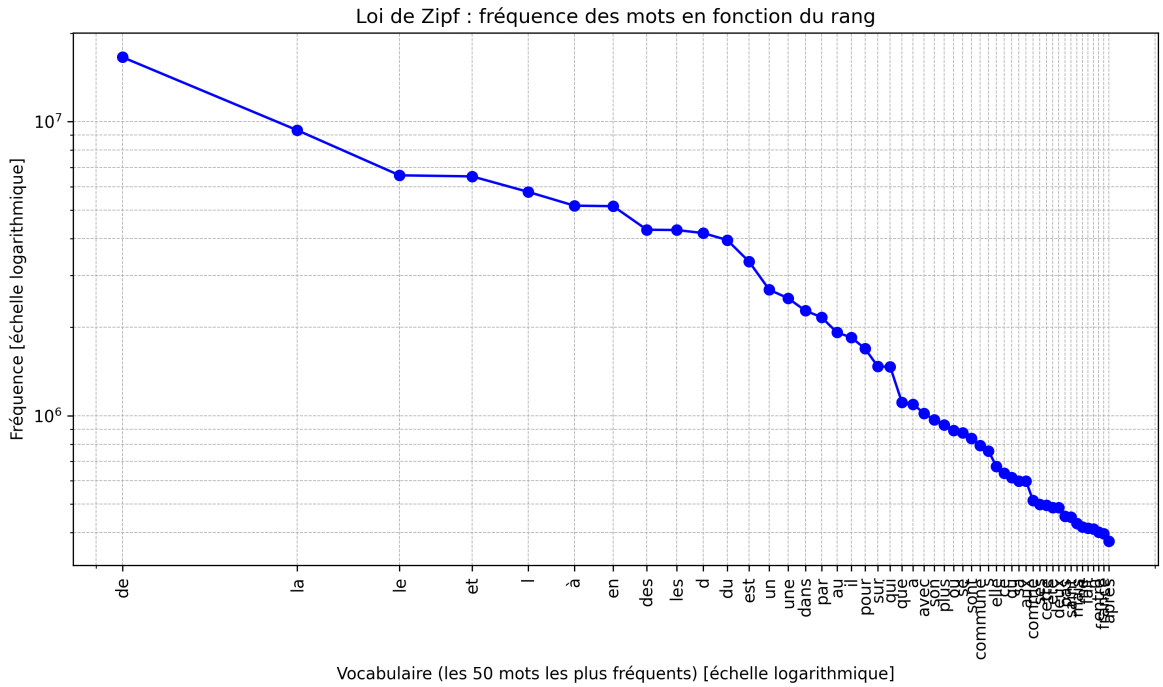
Vous trouverez dans le fichier `build_dictionary_from_wikipedia.py` un script qui construit un dictionnaire de mots à partir de Wikipédia. C'est voulu qu'il n'y ait pas de commentaires dans le code, votre tâche est de comprendre ce que fait le code. Si ce script vous semble trop lent sur votre machine, je vous conseille de lancer le code sur Colab.

Analyse du dictionnaire

Inspectez le haut (premiers 50 mots) du dictionnaire de fréquences des mots de Wikipédia (fichier `word_frequencies_2000.tsv`). Qu'en pensez-vous?

Une des particularités d'un langage naturel est que la fréquence des mots suit une loi de puissance. Cette loi est appelée loi de Zipf. Ainsi, les 50 mots les plus fréquents représentent environ 50% des mots utilisés dans le langage. Notez aussi qu'en utilisant une échelle logarithmique sur les deux axes, la courbe est presque une droite. Cela signifie que la fréquence des mots est inversement proportionnelle au rang du mot dans le dictionnaire[1].

[1] On dit que la fréquence des mots est une fonction de puissance (*power law*) du rang du mot dans le dictionnaire



Inspectez maintenant le bas du dictionnaire. Que voyez-vous?

```
héracléenne  3ō
iaidka      3
yoshikawa    3
octroyés     3
misumi       3
retraitées   3
bonzes       3
aétius       3
presbytére   3
peluche      3
ypres        3
tekle        3
```

On remarque des mots relativement rares de la langue française, comme par exemple `presbytére` ou `peluche`. On remarque aussi des noms propres, comme `misumi` ou comme `ypres`.

Notre dictionnaire contient également des fautes d'orthographe, comme `categorie` (sans l'accent) et des mots étrangers comme `accountability`.

Le bas de notre fichier (en anglais: **"long tail"**) se réfère à la partie de la distribution des fréquences où se trouvent les mots qui apparaissent très rarement. Ces mots sont peu fréquents, mais ils sont nombreux et représentent une part significative du vocabulaire total d'une langue. Cela inclut des mots rares mais corrects, ainsi que des erreurs de frappe potentielles.

Correction d'un mot

Nous allons maintenant utiliser notre dictionnaire pour corriger les fautes d'orthographe.

Commencez par écrire une fonction `load_dictionary(filename: str)-> dict[str, int]` qui charge le dictionnaire de fréquences des mots à partir de notre fichier `word_frequencies_2000.tsv`.

Ensuite, écrivez une fonction `correct_spelling(mot: str)-> str` qui prend en entrée un mot et qui retourne la meilleure correction de ce mot. Votre fonction utilisera votre dictionnaire de fréquences des mots. Elle devra retourner le mot qui a la fréquence la plus élevée parmi les mots de l'ensemble des édits de `mot`.

Finalement, testez votre fonction avec le mot `"chaussetes"` et aussi avec le mot `"chausssettes"`:

```
print(correct_spelling("chaussetes"))     # attendu: "chaussettes"
print(correct_spelling("chausssettes"))   # attendu: "chaussettes"
```

Partie 4: Evaluation

Dans la pratique, on développe des versions successivement plus performantes d'un algorithme (dans notre cas, un correcteur orthographique). La grande question est de savoir si nos modifications améliorent la qualité de la correction (ou la péjorent…). Pour cela, il nous faut un moyen de *mesurer la qualité de la correction*. On peut bien sûr tester avec quelques mots, comme nous l'avons fait ci-dessus, mais ce qu'il nous faudrait vraiment est un **moyen *automatique* de mesurer la qualité de la correction pour un *grand nombre de mots***. Pour cela, on utilisera un *corpus d'évaluation*.

Corpus d'évaluation

Regardez le fichier `evaluation_corpus.tsv`. Il contient une centaine de mots (un par ligne, avec le mot incorrect en première colonne et le mot correct en deuxième colonne). Ce corpus a été créé manuellement. Sa taille est relativement petite, mais elle est suffisante pour nous donner une petite idée de la qualité de la correction de notre correcteur orthographique.

```
acceuil     accueil
apartheid   apartheid
apparament  apparemment
aréoport    aéroport
ballade     balade
[...]
```

Exécutez le script `evaluate_spelling_correction.py`. Ce script affiche le mot à corriger, le mot correct, la proposition de correction de votre correcteur orthographique et un booléen selon si la correction est correcte ou non. Ce script calcule également l'accuracy de votre correcteur orthographique sur l'ensemble du corpus (c'est-à-dire le nombre de mots correctement corrigés divisé par le nombre total de mots. Allez regarder sur Google ce qu'est l'accuracy). Comment interprétez-vous l'accuracy de votre correcteur orthographique? Est-ce une bonne performance?

Partie 5: Amélioration

WARNING: n'utilisez pas de LLM pour la partie ci-dessous. Un autre labo reprendra l'algorithme de correction et vous devrez l'améliorer en vous aidant des LLMs.

Votre 1ère version de correcteur orthographique est une bonne base pour commencer (on appelle ça une *baseline*). Maintenant, le but est de l'améliorer. Voici quelques pistes d'amélioration que vous pouvez réaliser:

- prendre en compte les accents

- utiliser un dictionnaire plus grand (par exemple, avec 200'000 articles de Wikipédia), ce qui nous permettrait d'avoir des mots plus rares dans le dictionnaire. Quels sont les coûts - bénéfices?
- autoriser 2 erreurs (2 edits), ce qui nous permettrait de corriger des mots comme `"sufisament"` en `"suffisamment"` (2 edits: ajouter un "f" et ajouter un "m"). Idem: quels sont les coûts - bénéfices?

Après chaque modification, exécutez le script `evaluate_spelling_correction.py` pour évaluer les performances de votre correcteur orthographique.